

DP procedura za iskaznu logiku

Nikola Kuburović, 1048/2024

21. jul 2026.

Sažetak

DP (Davis-Putnam) procedura za iskaznu logiku ispituje zadovoljivost date formule iskazne logike. Sastoji se iz tri koraka:

- Unit propagation
- Pure literal
- Variable elimination

pri čemu je glavni korak upravo Variable elimination, dok prva dva služe da se poboljšaju performanse algoritma.

1 Uvod

Jedan od velikih problema u Automatskom rezonovanju jeste problem ispitivanja *zadovoljivosti* formule: za datu formulu, neophodno je ispitati da li postoji dodela istinitosnih vrednosti njenim promenljivim, takva da je formula tačna.

U našem slučaju, u pitanju je problem ispitivanja zadovoljivosti formule iskazne logike.

Davis-Putnam (DP) procedura, kojom se ovaj rad bavi, je jedna od najranijih procedura za ispitivanje zadovoljivosti logičke formule, i nju su 1960. godine predložili Martin Dejvis i Hilari Putnam [1]

Interesantno je da je originalni DP algoritam zapravo namenjen za ispitivanje zadovoljivosti formule logike prvog reda, i oslanja se na Herbrandovu teoremu i ideju da se problem u logici prvog reda svede na ispitivanje niza iskaznih formula, pri čemu se za te iskazne formule koristi upravo DP procedura koju ovaj rad opisuje.

Ostatak ovog rada organizovan je na sledeći način. U poglavlju 2 navodimo osnovne definicije, pojmove i notaciju koja će biti korišćena u ostatku rada. U poglavlju 3 dajemo detaljni opis algoritma koji je predmet ovog rada. Poglavlje 4 sadrži detalje implementacije metode. Najзад, poglavlje 5 sadrži zaključna razmatranja i osvrt na metodu opisanu u radu.

2 Osnove

Formule iskazne logike. Grade se od skupa iskaznih *promenljivih* (atoma) $p, q, r, \dots$, logičkih konstanti $\top$ (tačno) i $\perp$ (netačno), i logičkih veznika $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$.

Valuacija. Preslikavanje v koje svakoj promenljivoj dodeljuje istinitosnu vrednost iz $\{\top, \perp\}$; ono se na uobičajen način proširuje na sve formule.

Zadovoljivost. Za formulu F kažemo da je *zadovoljiva* ako postoji valuacija v takva da je $v(F) = \top$; u suprotnom je *nezadovoljiva*.

Tautologija. Formula je *tautologija* ako je tačna pri svakoj valuaciji. *SAT problem* jeste problem odlučivanja da li je data formula zadovoljiva.

Literal. Promenljiva (p) ili njena negacija $(\neg p)$.

Klauza. Disjunkcija literala, npr. $(p \vee \neg q \vee r)$.

Konjunktivna normalna forma. Formula je u *konjunktivnoj normalnoj formi* (KNF) ako je konjunkcija klauza. DP procedura radi isključivo nad formulama u KNF.

Uslovi zaustavljanja. Uvodimo dve granične vrednosti koje čine uslove zaustavljanja algoritma:

- **Prazna klauza** $\{\}$ predstavlja $\perp$: disjunkcija bez članova je netačna. Pojava prazne klauze znači da je formula **nezadovoljiva**.
- **Prazan skup klauza** $\{\}$ predstavlja $\top$: konjunkcija bez članova je tačna. Prazan skup klauza znači da je formula **zadovoljiva**.

Rezolucija. Ako klauza C_1 sadrži literal x , a klauza C_2 literal $\neg x$, njihova *rezolventa* po *pivotu* x je klauza

$$R = (C_1 \setminus \{x\}) \cup (C_2 \setminus \{\neg x\}).$$

Rezolventa je logička posledica konjunkcije $C_1 \wedge C_2$. Rezolucija je pravilo izvođenja na kome počiva glavni korak DP procedure.

Tseytin transformacija. Proizvoljna formula ne mora biti u KNF, a naivno prevođenje može eksponencijalno povećati broj klauza. *Tseytin transformacija* zaobilazi taj problem tako što svakoj podformuli dodeljuje novu promenljivu i dodaje klauze koje iskazuju ekvivalenciju te promenljive sa odgovarajućom podformulom. Rezultat je formula u KNF koja je *ekvizadovoljiva* sa polaznom (zadovoljiva ako i samo ako je polazna zadovoljiva), uz linearno povećanje veličine. U našoj implementaciji Tseytin transformacija je ulazni korak koji formulu prevodi u KNF pre pokretanja DP procedure.

3 Opis metode

DP procedura ponavlja 3 glavna koraka dok ne dođe do odluke o tome da li je formula zadovoljiva ili ne:

3.1 Unit propagation

Jedinična klauza je klauza sa tačno jednim literalom l . Da bi cela formula bila tačna, taj literal mora biti tačan. Zato:

- svaka klauza koja sadrži l je zadovoljena i uklanja se iz formule;
- iz svake klauze koja sadrži $\neg l$ uklanja se literal $\neg l$ (jer ne doprinosi istinitosti klauze).

Uklanjanje $\neg l$ može neku klauzu svesti na jediničnu, pa ponavljamo isti postupak, ili na praznu klauzu, a tada je formula **nezadovoljiva**. Takođe, moguće je i da uklanjanjem klauza dobijemo prazan skup klauza, i tada zaključujemo da je formula **zadovoljiva**.

3.2 Pure literal

Literal l je *čist* (eng. *pure*) ako se u celoj formuli pojavljuje samo u jednom polaritetu tj. $\neg l$ se ne javlja ni u jednoj klauzi. Tada uvek možemo tom literalu dodeliti vrednost koja ga čini tačnim, ne narušavajući nijednu drugu klauzu. Sve klauze koje sadrže l postaju zadovoljene i uklanjaju se. Ovaj korak može uzrokovati zadovoljivost (ispražnjenjem skupa klauza), ali nikada nezadovoljivost, jer samo briše cele klauze, a nikada ne skraćuje klauzu, pa ne može stvoriti praznu klauzu (slučaj koji dovodi do UNSAT zaključka).

3.3 Variable elimination

Ovo je glavni, najvažniji korak DP procedure. Bira se *pivot* promenljiva x i skup klauza S se podeli na tri dela:

- $P ::$ klauze koje sadrže x ;
- $N ::$ klauze koje sadrže $\neg x$;
- $R ::$ ostale klauze (ne sadrže x).

Zatim se generišu sve rezolvente po pivotu x : za *svaki* par $(C_p, C_n) \in P \times N$ računa se rezolventa. Novi skup klauza je

$$S' = R \cup \{ \text{rezolventa}(C_p, C_n) \mid C_p \in P, C_n \in N \},$$

uz izbacivanje rezolventi koje su *tautologije* (sadrže i y i $\neg y$ za neku promenljivu y , pa su trivijalno tačne). Skup S' ne sadrži više promenljivu x i *ekvizadovoljiv* je sa S (predstavlja $\exists x. S$). Uslovi zaustavljanja su oni opisani u paragrafu Uslovi zaustavljanja:

- ako neka rezolventa bude prazna klauza $\Rightarrow$ **Formula je nezadovoljiva**;
- ako S' bude prazan skup klauza $\Rightarrow$ **Formula je zadovoljiva**.

Cena koraka i izbor pivota. Eliminacija promenljive sa $|P| = a$ i $|N| = b$ pojavljivanja *uklanja* $a + b$ klauza, ali *dodaje* do $a \cdot b$ rezolventi, po jednu za svaki par iz Dekartovog proizvoda $P \times N$. Kada je $a \cdot b > a + b$, broj klauza raste, i to se kroz uzastopne eliminacije može akumulirati eksponencijalno. Ovo je razlog za nastanak *DPLL procedure* par godina kasnije. Kako je broj generisanih rezolventi $a \cdot b$, mogli bismo da koristimo heuristiku za izbor pivota koja *minimizuje* $a \cdot b$. Pronalaženje optimalnog redosleda eliminacije je NP-težak problem.

Napomena: Naša implementacija ne koristi nikakvu heuristiku za odabir pivota, već najprostije samo uzme prvi dostupan literal iz prve klauze u skupu klauza, i tako u svakoj iteraciji.

4 Implementacija

Metoda je implementirana u programskom jeziku **C++** (standard C++23). Kod je organizovan u sledeće celine:

- `lib.h / lib.cpp` :: osnovni tipovi podataka i Tseytin transformacija;
- `dp.h / dp.cpp` :: implementacija DP procedure i njena tri koraka;
- `main.cpp` :: konstrukcija ulazne formule, poziv procedure i ispis rezultata.

4.1 Reprezentacija podataka

Ulazna formula predstavlja se apstraktnim sintaksnim stablom preko `std::variant` tipa `Formula` (koji može biti `False`, `True`, `Atom`, `Not`, `Binary`). Prvo se koristi Tseytin transformacija da bi se formula svela na KNF oblik, jer to zahteva DP procedura. Za KNF koristimo sledeće tipove:

```
struct Literal { bool pos; std::string name; };
using Clause    = std::vector<Literal>;
using NormalForm = std::vector<Clause>;
```

`Literal` nosi ime promenljive i polaritet (`pos`). Nad tipom `Literal` definisani su operatori `==` i `<` (poređenje leksikografski po imenu, pa po polaritetu). Operator `<` je neophodan da bi klauze i skup klauza mogli da se porede, a to koristimo u procesu deduplikacije, kao i za korišćenje u `std::set`.

4.2 Ključne funkcije

Povratni tip koraka. Svaki od koraka koji može doneti konačan zaključak vraća `std::variant<NormalForm, bool>`: `bool` kodira ishod ispitivanja zadovoljivosti (`true` = SAT, `false` = UNSAT), dok `NormalForm` znači „nema zaključka, nastavi sa uprošćenom formulom”.

- `perform_unit_propagation(NormalForm&)` :: u petlji pronalazi jediničnu klauzu (`std::ranges::find_if` po uslovu `size() == 1`), uklanja klauze koje sadrže literal (`std::erase_if`) i skraćuje klauze koje sadrže suprotni literal. Funkcija vraća `false` ako neka klauza postane prazna, ili `true` ako skup klauza (tj. formula) postane prazan, a ako nije moguće zaključiti ni jedno ni drugo, vraća formulu za dalju obradu.

- `pure_literal(NormalForm&)` :: gradi mapu broja pojavljivanja literala po polaritetu (negativni se razlikuju po tome što imaju prefiks "!"), pronalazi literal čiji suprotni polaritet ne postoji i uklanja sve klauze koje ga sadrže, i to ponavlja sve dok ima čistih literala.
- `elimination(NormalForm&, const Literal& pivot)` :: particioniše klauze u `std::set<Clause> P, N, R`, generiše rezolvente za sve parove iz $P \times N$, preskače tautologije (`does_contain_tautology`), vraća `false` na praznu rezolventu i `true` na prazan S' , pri čemu je S' ekvizadovoljiva formula formuli S sa početka koraka.
- `dp(NormalForm&)` :: glavna petlja koja poziva tri koraka redom. Kao pivot bira prvi literal prve klauze (`cnf[0][0]`), a pošto svaka iteracija eliminiše po jednu promenljivu, ovaj jednostavni izbor ne narušava korektnost.

Pomoćne funkcije.

- `negate_literal` obrće polaritet,
- `does_contain_tautology` proverava da li klauza sadrži komplementaran par literala,
- šablonska funkcija `deduplicate` uklanja duplikate (`sort + unique + erase`) i primenjuje se i unutar klauze (npr. rezolventa $p \vee p$) i nad skupom klauza.

4.3 Poznata ograničenja

Implementacija se više fokusira na jasnoću, umesto na maksimalnu efikasnost. Neki koraci (npr. ponovno računanje mape pojavljivanja u `pure_literal` u svakoj iteraciji, ili $O(n)$ pretrage po klauzama) mogli bi se ubrzati. Izbor pivota je trivijalan (prvi literal prve klauze) umesto heuristike koja minimizuje proizvod pojavljivanja tog literala u pozitivnom i negativnom polaritetu, što ne utiče na korektnost, ali može uticati na broj generisanih klauza.

4.4 Prevođenje i pokretanje

Potreban je kompajler sa punom podrškom za C++23 (npr. `g++ ≥ 14` ili `clang++ ≥ 18`), jer se koriste `std::ranges::contains`, `std::string::contains`, `std::vector::append_range` i `std::unreachable`. Ostatak koda oslanja se na `std::ranges` algoritme i `std::erase_if`, koji su dostupni od C++20.

Instrukcije za instalaciju potrebnih alata zavise od konkretne Linux distribucije na kojoj se pokreće program. Autor je koristio NixOS distribuciju, pa će instrukcije za nju specifično biti date u sekciji Pokretanje pomoću Nix-a i `devenv`-a. *Napomena:* `nix` menadžer paketa i `devenv` rade na bilo kojoj Linux distribuciji, i instrukcije za njihovo instaliranje zavise od konkretne distribucije koja se koristi.

Program se prevodi i pokreće na sledeći način:

```
cmake -S . -B build
cmake --build build
./build/main
```

Alternativno, direktno prevodiocem:

- pomoću g++:

```
g++ -std=c++23 main.cpp lib.cpp dp.cpp -o main
./main
```

- pomoću clang++:

```
clang++ -std=c++23 main.cpp lib.cpp dp.cpp -o main
./main
```

Program ispisuje KNF ulazne formule i konačan rezultat (SAT ili UNSAT).

4.5 Pokretanje pomoću Nix-a i devenv-a

Projekat sadrži `devenv.nix` konfiguraciju koja obezbeđuje kompletno razvojno okruženje (C++ lanac alata, CMake i pomoćne alate), pa nije potrebno ručno instalirati kompajler. Ukoliko su na mašini dostupni *Nix* i *devenv*, dovoljno je ući u okruženje:

```
devenv shell
```

Time se dobija ljuska u kojoj su svi potrebni alati na PATH-u. Prevođenje i pokretanje se zatim izvode uobičajenim CMake koracima:

```
cmake -S . -B build
cmake --build build
./build/main
```

Alternativno, pošto projekat sadrži i `.envrc` fajl, uz instaliran *direnv* okruženje se aktivira automatski pri ulasku u direktorijum projekta (nakon jednokratnog `direnv allow`), bez potrebe za ručnim pozivom `devenv shell`.

5 Zaključak

Literatura

- [1] Davis, Martin, and Hilary Putnam. *A computing procedure for quantification theory*. Journal of the ACM (JACM) 7.3 (1960): 201–215.